

S02- MultiLingual:

Response:

```
{
  "id": "chatcmpl-6ce4dd6e-b989-4342-8986-095311c35df8",
  "object": "chat.completion",
  "created": 1781631132,
  "model": "llama-3.3-70b-versatile",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "வணக்கம்! நன்றி மிக்க வரவேற்புக்கு நன்றி! இந்த API ஐ பயன்படுத்துவதற்கு, கீழே உள்ள படிகளைப் பின்பற்றவும்:\n\n1. **API கீ பெறு**:  
முதலில், நீங்கள் ஒரு API கீயைப் பெற வேண்டும். அதற்காக, நீங்கள் அதனை வழங்கும் தளத்தின் இணையதளத்திற்குச் செல்லவும், அங்கு பதிவு செய்து, API கீயைப் பெறுங்கள்.\n\n2. **API ஆவணங்களைப் படியுங்கள்**:  
ஒரு முறை நீங்கள் API கீயைப் பெற்ற பிறகு, அந்த தளத்தின் API ஆவணங்களைப் படிக்கவும். அவை உங்களுக்கு API எவ்வாறு செயல்படுகிறது, என்ன வகையான கோரிக்கைகளை அனுப்ப முடியும், மற்றும் எத்தகைய பதில்களை எதிர்பார்க்கலாம் என்பதைப் புரிந்துகொள்ள உதவும்.\n\n3. **API கோரிக்கைகளை அனுப்பு**:  
பின்னர், நீங்கள் தேவையான தரவுகளைப் பெறுவதற்காக அல்லது தரவுகளை சேமிக்கவும் அல்லது புதுப்பிக்கவும் API கோரிக்கைகளை அனுப்பத் தொடங்கலாம். இது பொதுவாக HTTP கோரிக்கைகள் மூலம் செய்யப்படுகிறது, அதாவது GET, POST, PUT, DELETE போன்றவை.\n\n4. **பதில்களை செயலாக்கு**:  
ஒரு முறை நீங்கள் API கோரிக்கையை அனுப்பிவிட்டால், அதற்கான பதிலைப் பெறுவீர்கள். இந்த பதில் பெரும்பாலும் JSON அல்லது XML போன்ற தரவு வடிவத்தில் இருக்கும். நீங்கள் இந்த பதிலை செயலாக்க வேண்டும், அதாவது தேவையான தகவல்களை பிரித்தெடுப்பது, பிழைகளைக் கண்டறிதல் போன்றவை.\n\n5. **பிழைகளைக் கண்டறிதல் மற்றும் சரி செய்தல்**:  
ஒரு API உடன் பணிபுரியும்போது, பிழைகள் ஏற்படலாம். அதனால், நீங்கள் பிழைகளைக் கண்டறிதல் மற்றும் சரி செய்தல் ஆகியவற்றிற்கான ஏற்பாடுகளைச் செய்ய வேண்டும். இதில் பிழை குறியீடுகளைப் பயன்படுத்துதல், பிழை செய்திகளைப் பகுப்பாய்வு செய்தல் போன்றவை அடங்கும்.\n\n6. **பாதுகாப்பு**:  
இறுதியாக, API ஐ பயன்படுத்தும்போது, பாதுகாப்புக்கு முக்கியத்துவம் அளிக்கவும். உங்கள் API கீயை பாதுகாப்பாகப் பயன்படுத்துதல், அனுப்பப்படும் தரவுகளை சுரக்குதல், மற்றும் தாக்குதல்களிலிருந்து பாதுகாப்பு போன்றவற்றை உறுதி செய்ய வேண்டும்.\n\nஇந்த படிகளைப் பின்பற்றுவதன் மூலம், நீங்கள் வெற்றிகரமாக API ஐ பயன்படுத்தலாம் மற்றும் உங்கள் செயலிக்கு தேவையான தரவுகளைப் பெறலாம். நன்றி!"
```

```

        "completion_tokens": 2286,
        "completion_time": 6.940825953,
        "total_tokens": 2419,
        "total_time": 6.962478518
    },
    "usage_breakdown": null,
    "system_fingerprint": "fp_dae98b5ecb",
    "x_groq": {
        "id": "req_01kv8qv1s4fw7regp4qh56c9zs",
        "seed": 1395092532
    },
    "service_tier": "on_demand"
}

```

```

{
    "id": "chatcmpl-8fbbbe071-5a91-429e-a4ab-0a04428d91f5",
    "object": "chat.completion",
    "created": 1781631269,
    "model": "openai/gpt-oss-120b",
    "choices": [
        {
            "index": 0,
            "message": {
                "role": "assistant",
                "content": "import io.restassured.RestAssured;\nimport io.restassured.http.ContentType;\nimport io.restassured.response.Response;\nimport io.restassured.specification.RequestSpecification;\nimport org.testng.Assert;\nimport org.testng.annotations.BeforeClass;\nimport org.testng.annotations.Test;\nimport java.util.HashMap;\nimport java.util.Map;\nimport java.util.Random;\n\npublic class PetStoreCrudTest {\n\n    private static final String BASE_URI = \"https://petstore.swagger.io/v2\";\n    private static long petId;\n    private static RequestSpecification requestSpec;\n\n    @BeforeClass\n    public void setUp() {\n        RestAssured.baseURI = BASE_URI;\n        requestSpec = RestAssured.given().contentType(ContentType.JSON)\n            .accept(ContentType.JSON);\n        petId = new Random().nextLong() & Long.MAX_VALUE;\n        // ensure positive\n    }\n\n    @Test(priority = 1)\n    public void createPet() {\n        Map<String, Object> pet = new HashMap<>();\n        pet.put(\"id\", petId);\n        pet.put(\"name\", \"Fluffy\");\n        pet.put(\"status\", \"available\");\n\n        Response response = requestSpec\n            .body(pet)\n            .when()\n            .post(\"/pet/\")\n            .then()\n            .extract()\n            .response();\n\n        Assert.assertEquals(response.getStatusCode(), 200, \"Create pet status code\");\n        Assert.assertEquals(response.jsonPath().getLong(\"id\"), petId, \"Created pet\");

```

```

ID\");\n        Assert.assertEquals(response.jsonPath().getString(\"name\"),
\"Fluffy\", \"Created pet
name\");\n        Assert.assertEquals(response.jsonPath().getString(\"status\"),
\"available\", \"Created pet status\");\n    }\n\n    @Test(priority = 2)\n    public
void getPetById() {\n        Response response =
requestSpec\n            .when()\n                .get(\"/pet/{petId}\",
petId)\n            .then()\n                .extract()\n                    .response
();\n\n        Assert.assertEquals(response.getStatusCode(), 200, \"Get pet status
code\");\n        Assert.assertEquals(response.jsonPath().getLong(\"id\"), petId,
\"Fetched pet
ID\");\n        Assert.assertEquals(response.jsonPath().getString(\"name\"),
\"Fluffy\", \"Fetched pet
name\");\n        Assert.assertEquals(response.jsonPath().getString(\"status\"),
\"available\", \"Fetched pet status\");\n    }\n\n    @Test(priority = 3)\n    public
void updatePet() {\n        Map<String, Object> updatedPet = new
HashMap<>();\n        updatedPet.put(\"id\", petId);\n        updatedPet.put(\"name\",
\"FluffyUpdated\");\n        updatedPet.put(\"status\",
\"pending\");\n\n        Response response =
requestSpec\n            .body(updatedPet)\n                .when()\n                    .put(\"/pet\")\n            .then()\n                .extract()\n                    .response();\n\n        Assert.assertEquals(response.getStatusCode(), 200, \"Update
pet status code\");\n        Assert.assertEquals(response.jsonPath().getLong(\"id\"),
petId, \"Updated pet
ID\");\n        Assert.assertEquals(response.jsonPath().getString(\"name\"),
\"FluffyUpdated\", \"Updated pet
name\");\n        Assert.assertEquals(response.jsonPath().getString(\"status\"),
\"pending\", \"Updated pet status\");\n    }\n\n    @Test(priority = 4)\n    public
void deletePet() {\n        Response response =
requestSpec\n            .when()\n                .delete(\"/pet/{petId}\",
petId)\n            .then()\n                .extract()\n                    .response
();\n\n        Assert.assertEquals(response.getStatusCode(), 200, \"Delete pet status
code\");\n        Assert.assertTrue(response.asString().contains(String.valueOf(petId)
), \"Delete response contains pet ID\");\n    }\n}

```

"reasoning": "We need to produce Java code only, no markdown fences, no explanations. Must include imports, class, beforeclass, test methods using TestNG. Use RestAssured. Provide CRUD for Pet.\n\nWe'll need base URI: <https://petstore.swagger.io/v2>\n\nDefine Pet POJO maybe using Map. But can use JSON string.\n\nCreate a pet with random id. Use static variable petId.\n\nUse RequestSpecification.\n\nImplement createPet: POST /pet, body includes id, name, status.\n\nValidate status 200 and id matches.\n\nGet: GET /pet/{petId}\n\nUpdate: PUT /pet with changed name or status.\n\nDelete: DELETE /pet/{petId}\n\nAdd assertions using Hamcrest or TestNG assertions.\n\nWe'll import io.restassured.RestAssured, io.restassured.specification.RequestSpecification, io.restassured.response.Response, org.testng.annotations.* and org.testng.Assert.\n\nImplement @BeforeClass to set baseURI.\n\nLet's write code.\n\n"

```

    },
    "logprobs": null,
    "finish_reason": "stop"

```

```

    }
  ],
  "usage": {
    "queue_time": 0.283777986,
    "prompt_tokens": 396,
    "prompt_time": 0.019128978,
    "completion_tokens": 968,
    "completion_time": 2.013660168,
    "total_tokens": 1364,
    "total_time": 2.032789146,
    "completion_tokens_details": {
      "reasoning_tokens": 197
    }
  },
  "usage_breakdown": null,
  "system_fingerprint": "fp_ca5edfaab2",
  "x_groq": {
    "id": "req_01kv8qzc88f8cvajdrj97tmjrg",
    "seed": 1179523768
  },
  "service_tier": "on_demand"
}

```

S03-GenerateAPITestcasesFromSwaggerCode